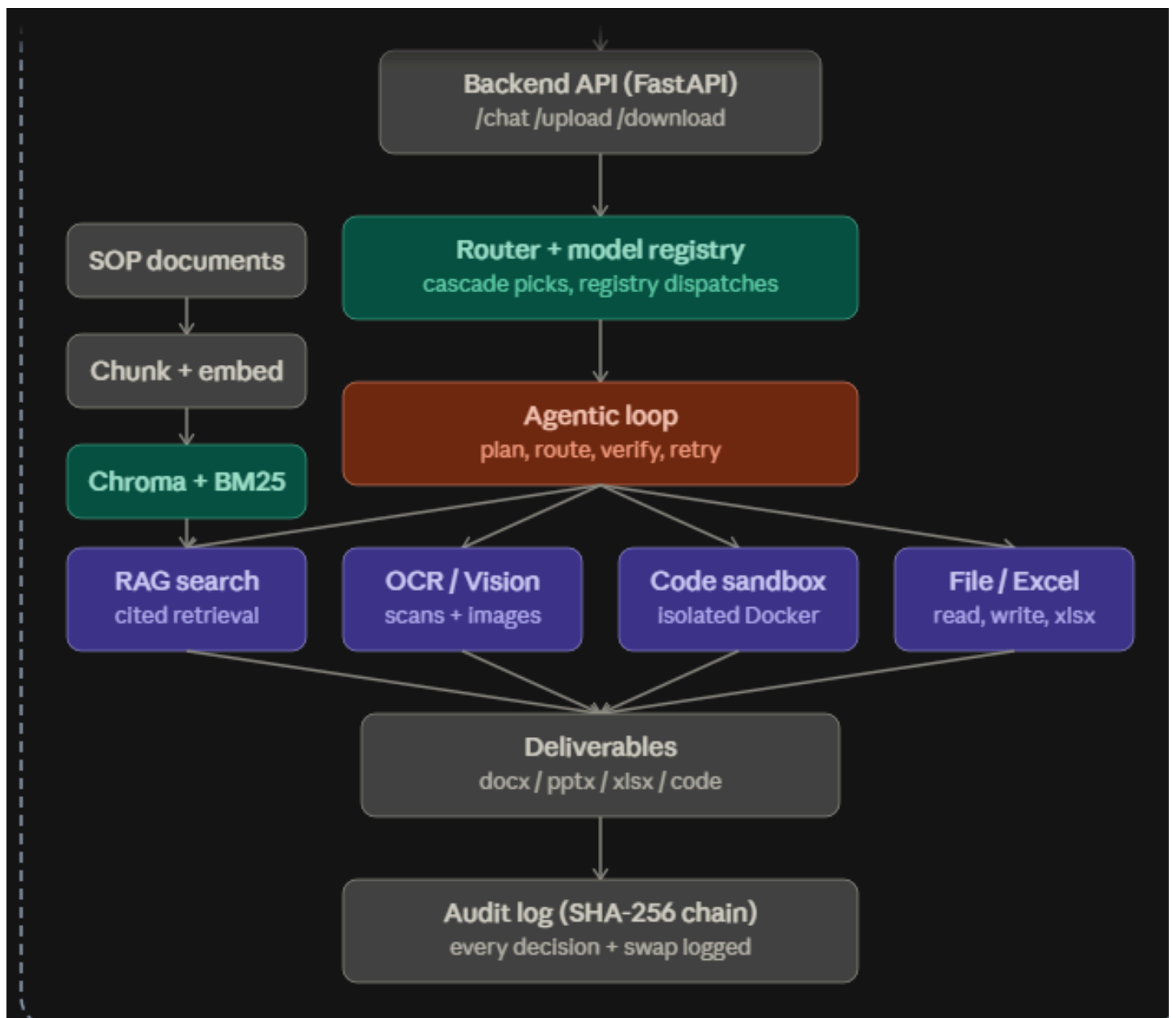




Backend API (FastAPI) — backend/main.py. Three endpoints only: POST /chat (router → agent loop → deliverable path), POST /upload, GET /download/{file_id}. Keep this thin — it's glue, not logic. That's what lets you swap the router or the loop later without touching this layer.

Router + model registry — models/. The cascade (semantic cache → regex → embedding classifier → LLM judge) picks a task_type; the registry resolves that to an actual model entry with VRAM footprint, quantization, and Ollama tag — qwen3-coder:30b for coding, qwen3-vl:8b for vision, a mid-size Qwen3 for reasoning, each served via ollama pull / ollama run locally, nothing hosted. The scheduler here keeps exactly one large model resident, LRU-evicting on idle.

Agentic loop — agent/. The planner decomposes the request into a subtask DAG, each subtask independently hits the router+registry above, and the confidence gate checks each result before accepting it — retry with the next-ranked model or a critic pass on low

confidence, hard-capped at 8-10 iterations so nothing loops forever. This is the block that turns "answered once and stopped" into something you can point at live.

RAG search / OCR-Vision / Code sandbox / File-Excel (tools row) — agent/tools.py, one function per tool behind the same run_tool(name, params) signature. rag_search calls the retrieval function below with source citations attached. OCR/Vision runs a fast Tesseract/PaddleOCR pass first, escalates to qwen3-vl:8b only on low-confidence or handwritten/diagram content. code_exec ships the coding model's output to the sandbox — --network none, CPU/memory limits, short timeout, non-root — and feeds stdout/stderr back in. spreadsheet_op is pandas/openpyxl against the uploaded file.

SOP documents → Chunk + embed → Chroma + BM25 (side branch) — rag/ingest.py and rag/retrieve.py. Offline pipeline, not part of the live request path: load your sample SOPs, chunk at ~500 tokens with overlap, embed with sentence-transformers/all-MiniLM-L6-v2, store in ChromaDB, and build a parallel BM25 index over the same chunks. Retrieval merges both and returns top-k with source citations — this is your existing hybrid-CRAG pattern, unchanged.

Deliverables — collects whatever the DAG produced and calls python-docx / python-pptx / openpyxl depending on task type, plus the raw code file for coding tasks. This is the block the PS explicitly wants as output, not chat text.

Audit log (SHA-256 chain) — every routing decision, model swap, confidence score, and escalation gets appended, each entry hashing in the previous entry's hash. Tamper with one line and the chain breaks — that's your live "prove it's tamper-proof" moment, not just an adjective on a slide.

The boundary — this isn't a code block, it's the constraint everything above respects: no line of code anywhere in this system makes an external call. network_monitor/ runs netstat/psutil or a visible Wireshark panel throughout the demo, and you test the whole thing with Wi-Fi physically off before demo day, not just claim it works that way.

I